

Fluence Coding Conventions

Team 20

Will Manicom, Sebastian Lassander, Ethan Batick,
Jason Riek

Introduction

This document provides coding standards and best practices for the Fluence project, built with a React frontend and a Python/Flask backend. Adherence to these standards is essential to ensure code quality, maintainability, and consistency across the full codebase.

General Principles

- **Readability and Clarity:** Code should be easily understandable. Prioritize clarity over cleverness.
- **Consistency:** The same coding conventions must be applied consistently across both the frontend and backend.
- **Simplicity and Performance:** Write simple and efficient code without premature optimization.
- **Commenting and Documentation:** Code should be well-commented and documented where necessary, especially around health data logic.

Formatting and Naming Conventions

Braces and Brackets (React/JS)

- Opening braces should not be on a new line.
- Always use braces for if, else, for, and while statements, even if the body contains only one statement.

Naming

Python (Flask)

- **Modules and packages:** Use all lowercase with underscores. Example: `health_record.py`.
- **Classes:** Use PascalCase. Example: `UserProfile`.
- **Functions and variables:** Use snake_case. Example: `get_heart_rate`.
- **Constants:** Use all uppercase letters with underscores. Example: `MAX_DATA_POINTS`.

React/JavaScript

- **Components:** Use PascalCase. Example: HeartRateChart.
- **Functions and variables:** Use camelCase. Example: fetchHealthData.
- **Constants:** Use all uppercase letters with underscores. Example: API_BASE_URL.
- **CSS class names:** Use kebab-case. Example: data-card-container.

Line Length

- Lines should not exceed **100 characters**. Break lines sensibly if necessary.

Comments and Documentation

- Inline comments should explain complex logic or health data processing decisions that are not immediately obvious.
- Avoid comments that state the obvious or provide no additional information.

Version Control

- Commit related changes in a logical unit and include a meaningful commit message.
- Use branches for features, bug fixes, and releases.
- All merges into main must go through a pull request, no direct commits to main.

Error Handling

Python/Flask

- Use exceptions rather than return codes for error handling.
- Return consistent JSON error responses from all API endpoints.
- Always clean up resources using with blocks or try/finally where applicable.

React

- Use error boundaries to gracefully handle component-level failures.
- Handle failed API calls explicitly, display user-friendly messages rather than silent failures.
- Never expose raw server error messages directly to the UI.

API Design

- All API endpoints should follow RESTful conventions.
- Use descriptive, resource-based URL paths. Example: GET `/api/users/{id}/metrics`.
- API responses should always include a consistent structure with a status indicator and data or error field.
- Validate all incoming request data on the Flask side before processing.

Code Reviews

- All code must be reviewed by at least one other team member before merging into the main branch.
- Reviewers should check for adherence to these standards, logical errors, and possible simplifications.
- Particular attention should be paid to any code that handles personal health data.

Security

- Adhere to best practices for secure coding to protect sensitive health data.
- Never store raw passwords, use a secure hashing library such as bcrypt.
- Sanitize and validate all user inputs on both the frontend and backend.
- Handle personal health information in accordance with applicable data protection regulations (e.g., HIPAA where relevant).
- Do not expose secret keys, API tokens, or credentials in the codebase, use environment variables.

Conclusion

This coding standards document establishes the foundation for developing the Fluence health data visualization project. All contributors are expected to adhere to these standards to maintain the quality and integrity of the codebase. Consistent application of these practices will enhance collaboration across the frontend and backend and support the overall success of the project.